You can choose to submit either A7 (worth 150 points), AD7 (worth 180), or both (worth 180) for 7th assignment you will get a score for this assignment either 150 points base or 180 points base.

Grading Criteria:

1. If the submitted program cannot be compiled or run on a Microsoft Visual Studio Community C++ compiler, the assignment will not get any credit.
2. If the file name does not follow the naming instruction as stated in syllabus, the assignment will get 5 to 15 points deduction.
3. If the header section of the submitted file does not include personal information and purpose of the file, the assignment will get 5 to 30 points deduction.
4. Late assignments will not get any credit for the assignment.
5. A submission does not follow the instructions of the assignment will get little credit.
6. A portion of the assignment is not finished or incomplete will get little or no credit.
7. Submit an assignment to a wrong drop box. The wrong drop box one will not get any credit.
8. If use A for AD or AD for A assignments in the file name, the student will not get any credit for the assignments.
9. If the submitted file(s) cannot be opened by the instructor's software (WinZip, Visual Studio, or text editor) the assignment will have little credit.
10. If the submission file does not include all necessary programming files to run the program, the program will get little credit.
11. If the program does not separate into several files (driver, header file, implementation file, etc.) as describe in the assignment, the program will get little credit.

12. If the displaying result of the program is not exactly match with the graphic in the assignment description or the displaying result of the executable file which comes with the assignment, the assignment will get little credit.
13. If the program does not have enough detail documentation in the programming area the program will get 5 to 30 points deduction.
14. If the program includes some unrelated documentation the program will get some points deduction.

A7

(Quicksort) The recursive sorting technique called quicksort uses the following basic algorithm for a one-dimensional vector of values:

a) Partitioning Step: Take the first element of the unsorted vector and determine its final location in the sorted vector (i.e., all values to the left of the element in the vector are less than the element's value, and all values to the right of the element in the vector are greater than the element's value—we show how to do this below). We now have one value in its proper location and two unsorted subvectors.

b) Recursion Step: Perform the Partitioning Step on each unsorted subvector. Each time the Partitioning Step is performed on a subvector, another value is placed in its final location of the sorted vector, and two unsorted subvectors are created. When a subvector consists of one element, that element's value is in its final location (because a one-element vector is already sorted).

The basic algorithm seems simple enough, but how do we determine the final position of the first element value of each subvector? As an example, consider the following set of values (the value in bold is for the partitioning element—it will be placed in its final location in the sorted vector):

37 2 6 4 89 8 10 12 68 45

Starting from the rightmost element of the vector, compare each element value with 37 until an element value less than 37 is found; then swap 37 and that element's value. The first element value less than 37 is 12, so 37 and 12 are swapped. The new vector is

12 2 6 4 89 8 10 **37** 68 45

Element value 12 is in italics to indicate that it was just swapped with 37. Starting from the left of the vector, but beginning with the element value after 12, compare each element value with 37 until an element value greater than 37 is found— then swap 37 and that element value. The first element value greater than 37 is 89, so 37 and 89 are swapped. The new vector is

12 2 6 4 **37** 8 10 89 68 45

Starting from the right, but beginning with the element value before 89, compare each element value with 37 until an element value less than 37 is found—then swap 37 and that element value. The first element value less than 37 is 10, so 37 and 10 are swapped. The new vector is

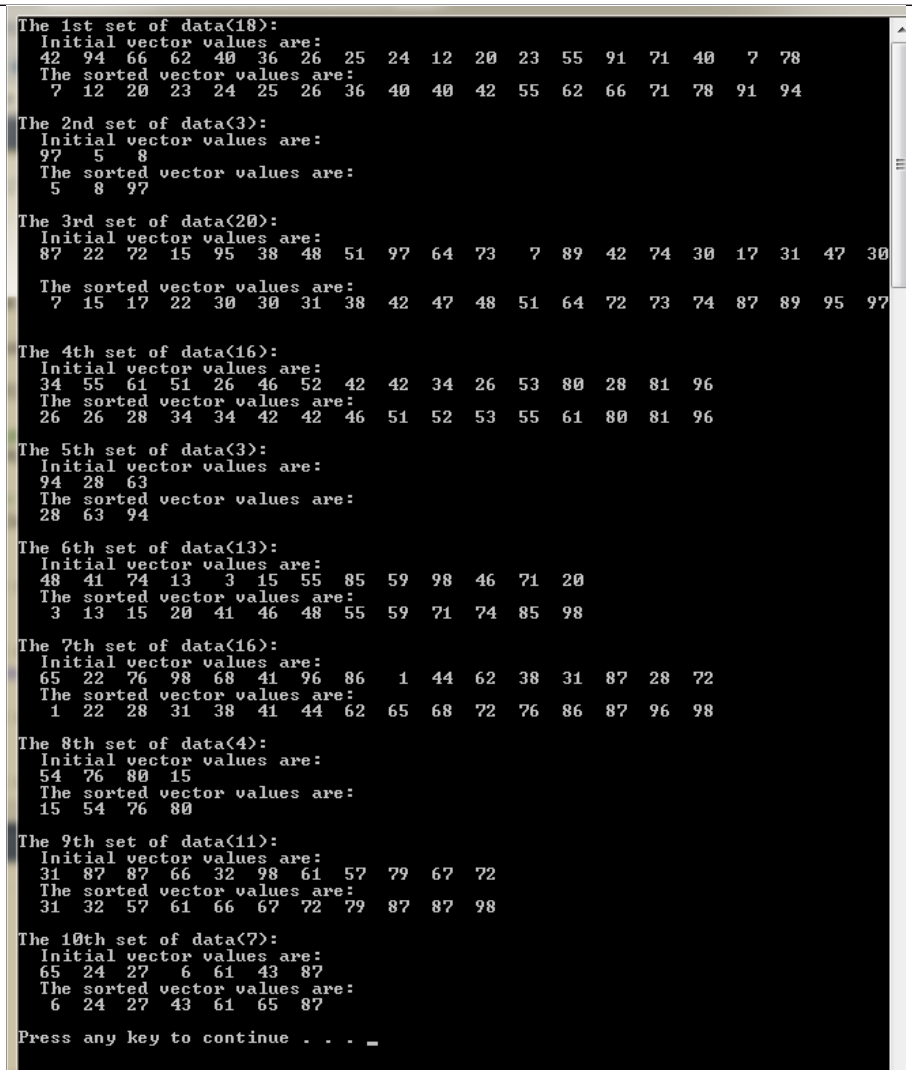
12 2 6 4 10 8 **37** 89 68 45

Starting from the left, but beginning with the element value after 10, compare each element value with 37 until an element value greater than 37 is found—then swap 37 and that element value. There are no more element values greater than 37, so when we compare 37 with itself, we know that 37 has been placed in its final location of the sorted vector. Every value to the left of 37 is smaller than it, and every value to the right of 37 is larger than it. Once the partition has been applied on the previous vector, there are two unsorted subvectors. The subvector with values less than 37 contains 12, 2, 6, 4, 10 and 8. The subvector with values greater than 37 contains 89, 68 and 45. The sort continues recursively, with both subvectors being partitioned in the same manner as the original vector.

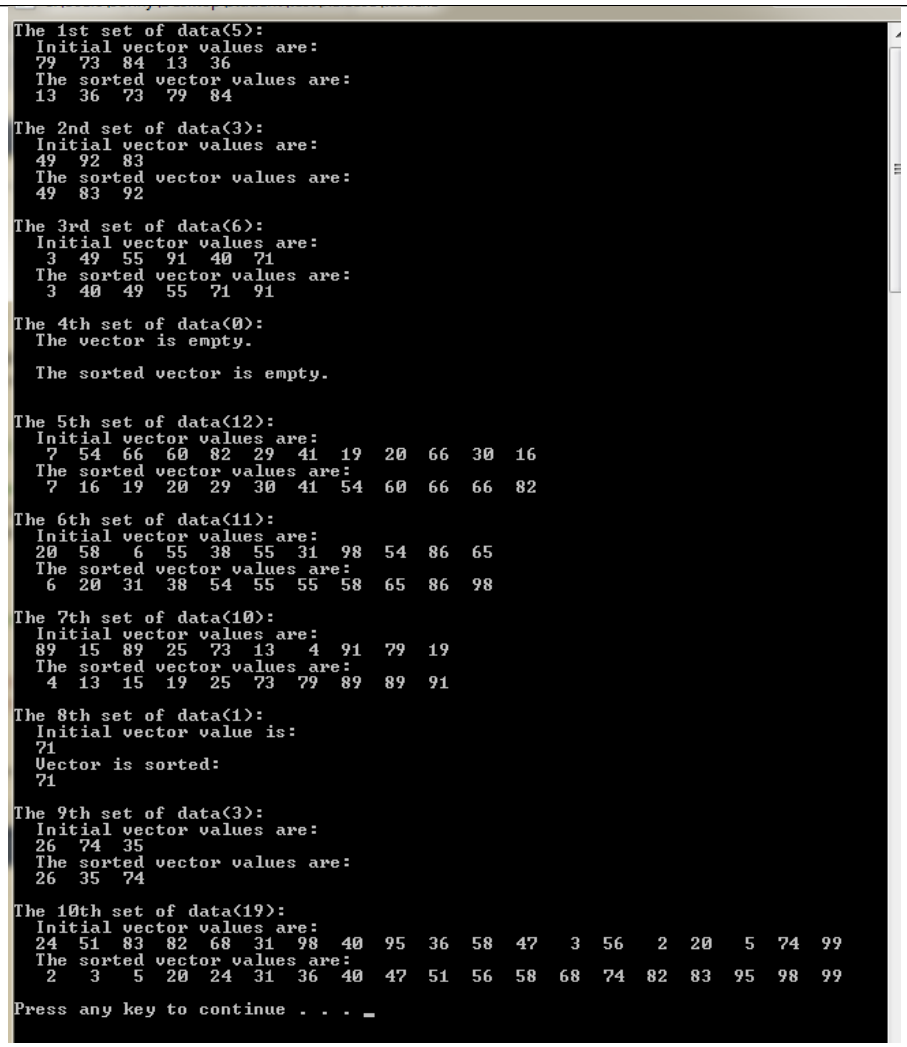
Based on the preceding discussion, write a recursive function, quickSortHelper, to sort ten one-dimensional integer vectors. The function should receive as arguments a starting index and an ending index on the original vector being sorted. Please create ten vectors with random size from 0 ~20 elements and randomly choose the size of the vector numbers from 0~100. Put the numbers in the vectors and use quick sort to sort them.

This assignment comes with a CISP400V10A7.zip file. It includes CISP400V10A7.exe file. The CISP400V10A7.exe file is an executable file. You can double click the file to get to the expecting result of this assignment.

The following are some of the displays of the expecting results.

Please be aware the vector size is randomly choose from 0 ~ 20 and the created sized for a vector is listed inside a pair of parentheses (“ ( )”).	

```
The 1st set of data(18):
  Initial vector values are:
  42  94  66  62  40  36  26  25  24  12  20  23  55  91  71  40   7  78
  The sorted vector values are:
   7  12  20  23  24  25  26  36  40  40  42  55  62  66  71  78  91  94

The 2nd set of data(3):
  Initial vector values are:
  97   5   8
  The sorted vector values are:
   5   8  97

The 3rd set of data(20):
  Initial vector values are:
  87  22  72  15  95  38  48  51  97  64  73   7  89  42  74  30  17  31  47  30

  The sorted vector values are:
   7  15  17  22  30  30  31  38  42  47  48  51  64  72  73  74  87  89  95  97


The 4th set of data(16):
  Initial vector values are:
  34  55  61  51  26  46  52  42  42  34  26  53  80  28  81  96
  The sorted vector values are:
  26  26  28  34  34  42  42  46  51  52  53  55  61  80  81  96

The 5th set of data(3):
  Initial vector values are:
  94  28  63
  The sorted vector values are:
  28  63  94

The 6th set of data(13):
  Initial vector values are:
  48  41  74  13   3  15  55  85  59  98  46  71  20
  The sorted vector values are:
   3  13  15  20  41  46  48  55  59  71  74  85  98

The 7th set of data(16):
  Initial vector values are:
  65  22  76  98  68  41  96  86   1  44  62  38  31  87  28  72
  The sorted vector values are:
   1  22  28  31  38  41  44  62  65  68  72  76  86  87  96  98

The 8th set of data(4):
  Initial vector values are:
  54  76  80  15
  The sorted vector values are:
  15  54  76  80

The 9th set of data(11):
  Initial vector values are:
  31  87  87  66  32  98  61  57  79  67  72
  The sorted vector values are:
  31  32  57  61  66  67  72  79  87  87  98

The 10th set of data(7):
  Initial vector values are:
  65  24  27   6  61  43  87
  The sorted vector values are:
   6  24  27  43  61  65  87

Press any key to continue . . .
```

Please be aware that the display of the vector size equals to zero and one are different from the size greater than and equal to 2.	

```
The 1st set of data(5):
  Initial vector values are:
  79  73  84  13  36
  The sorted vector values are:
  13  36  73  79  84

The 2nd set of data(3):
  Initial vector values are:
  49  92  83
  The sorted vector values are:
  49  83  92

The 3rd set of data(6):
  Initial vector values are:
   3  49  55  91  40  71
  The sorted vector values are:
   3  40  49  55  71  91

The 4th set of data(0):
  The vector is empty.

  The sorted vector is empty.


The 5th set of data(12):
  Initial vector values are:
   7  54  66  60  82  29  41  19  20  66  30  16
  The sorted vector values are:
   7  16  19  20  29  30  41  54  60  66  66  82

The 6th set of data(11):
  Initial vector values are:
  20  58   6  55  38  55  31  98  54  86  65
  The sorted vector values are:
   6  20  31  38  54  55  55  58  65  86  98

The 7th set of data(10):
  Initial vector values are:
  89  15  89  25  73  13   4  91  79  19
  The sorted vector values are:
   4  13  15  19  25  73  79  89  89  91

The 8th set of data(1):
  Initial vector value is:
  71
  Vector is sorted:
  71

The 9th set of data(3):
  Initial vector values are:
  26  74  35
  The sorted vector values are:
  26  35  74

The 10th set of data(19):
  Initial vector values are:
  24  51  83  82  68  31  98  40  95  36  58  47   3  56   2  20   5  74  99
  The sorted vector values are:
   2   3   5  20  24  31  36  40  47  51  56  58  68  74  82  83  95  98  99

Press any key to continue . . .
```

Please document all your files properly and zip all your files into a proper named zip file for this assignment (refer to the assignment section of the class syllabus) and submit it to the A7 folder of the Canvas Website.

Worth 150 points

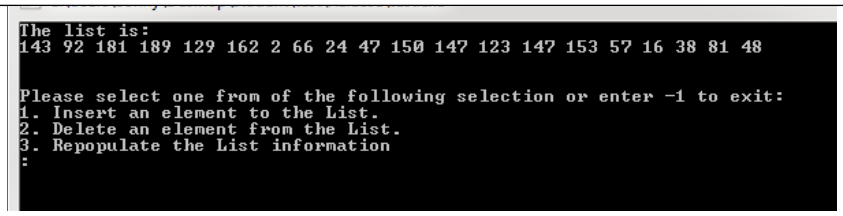
AD7

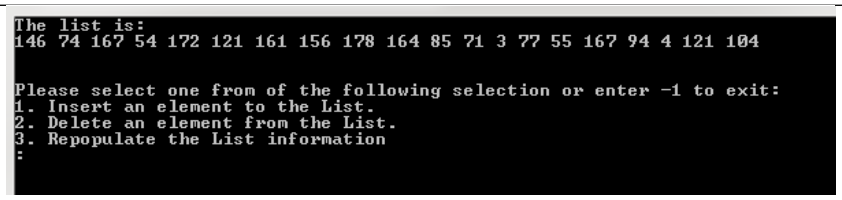
Our linked list class template allowed insertions and deletions at only the front and the back of the linked list. These capabilities were convenient for us when we used private inheritance and composition to produce a stack class template and a queue class template with a minimal amount of code by reusing the list class template. Actually, linked lists can do more than those provided by the textbook. In this assignment we want to have a linked list class template to handle insertions and deletions anywhere in the list.

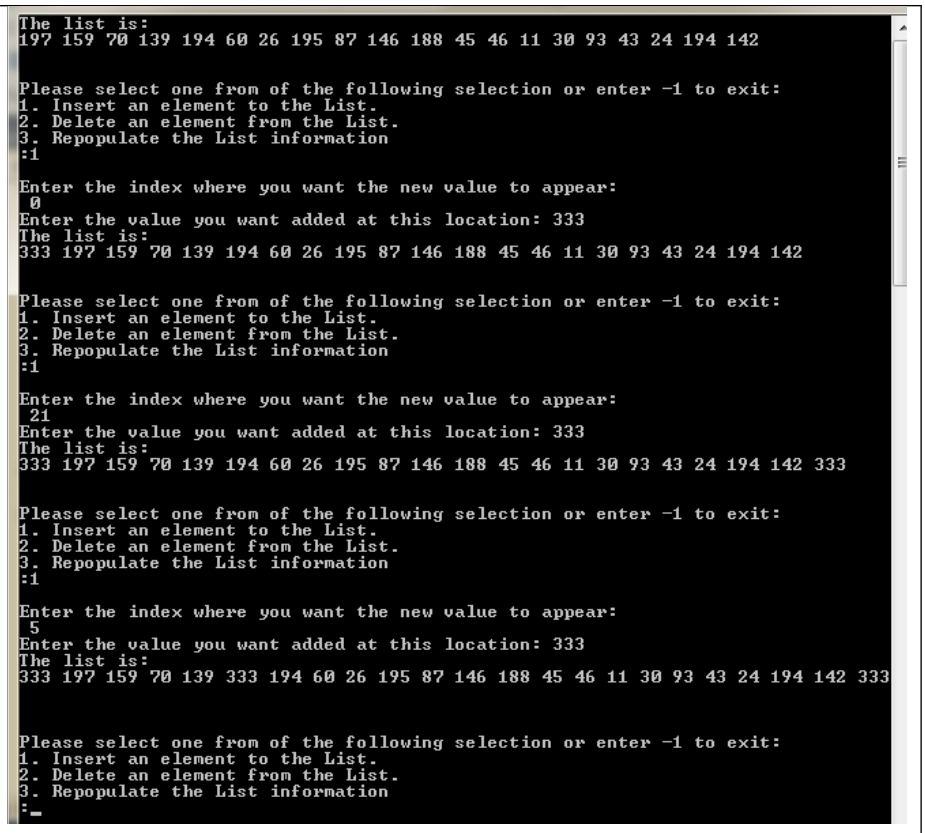
Modify ListNode.h(attached) and List.h(attached) to work with a driver to create an interface to add anywhere in the list , delete a particular element, repopulate the list and exit the list. The program will populate a 20 elements integer list with random number from 0 ~ 200. The list can expend to more than 100 and shrink to 0 element.

This assignment comes with a CISP400V10AD7.zip file. It includes CISP400V10AD7.exe file. The CISP400V10AD7.exe file is an executable file. You can double click the file to get to the expecting result (see the picture below) of this assignment. After you finish modifying ListNode.h and List.h and create CISP400V10AD7.cpp file you can put them in a project and run to the exe file result.

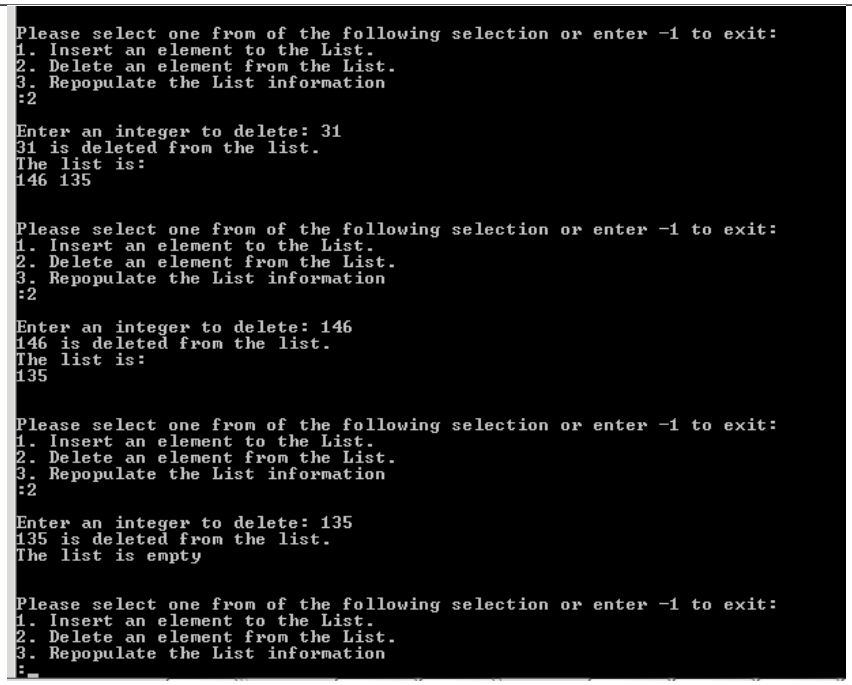
The following are some of the displays of the expecting results.

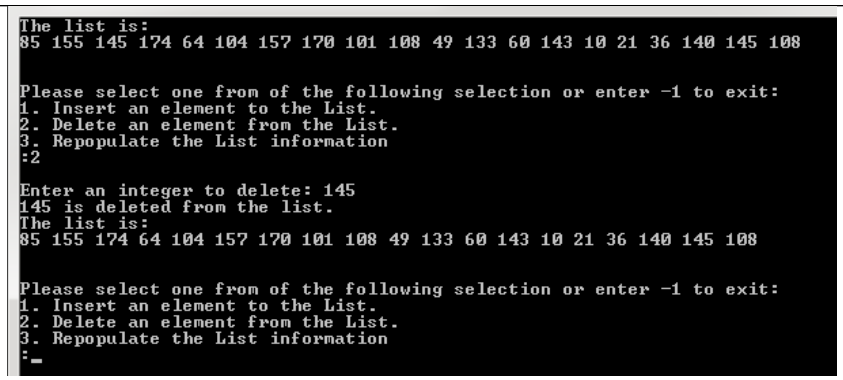
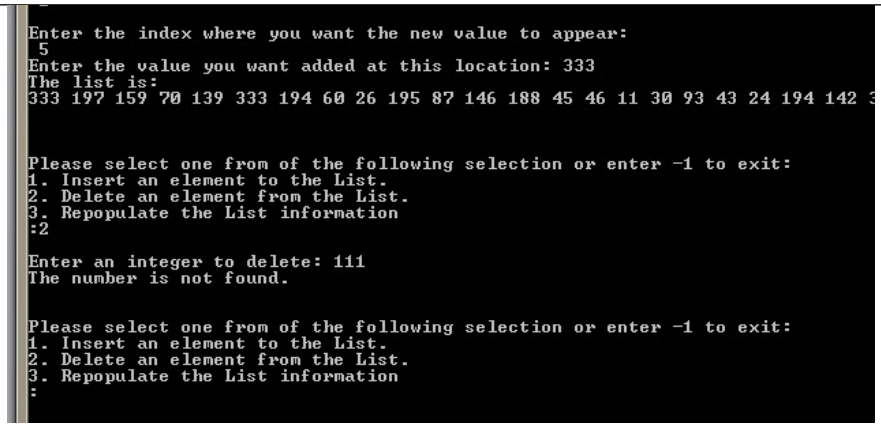
The right side of picture is a display of a starting of the program. The list contains 20 numbers which is in between 0	

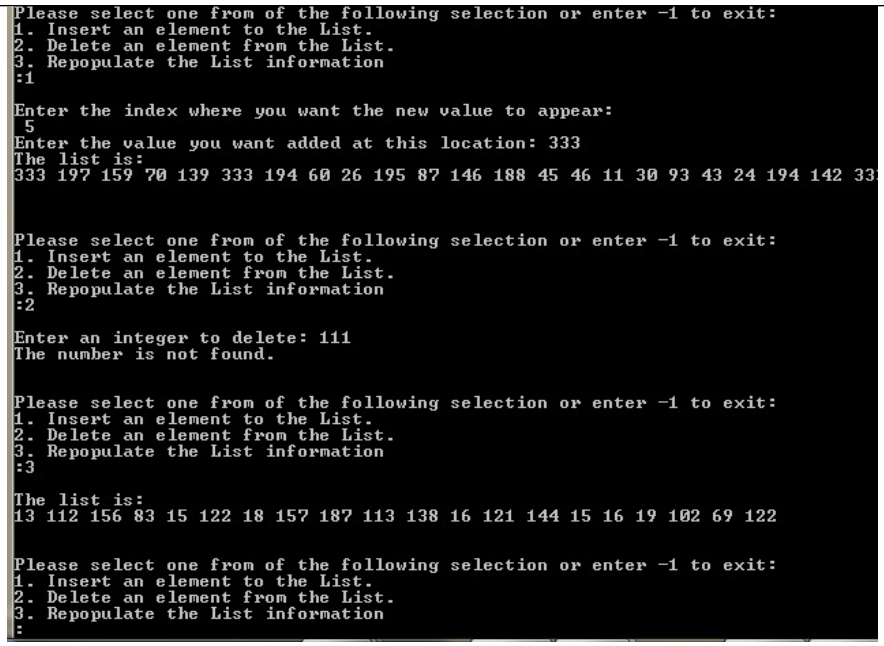
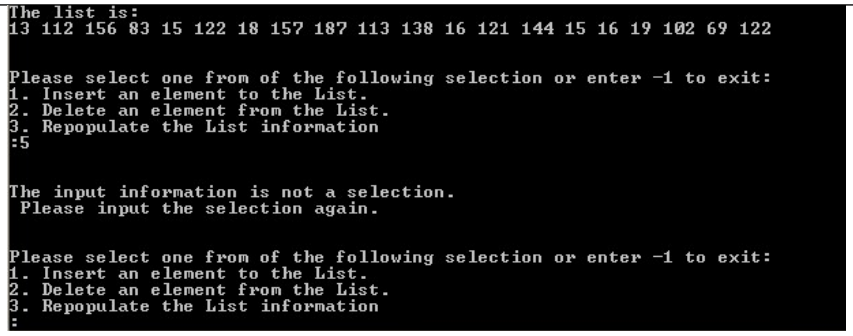
and 200.	
We should get a different list of data every time we start the program.	
We should be able to Exit the program any point when -1 is entered at a selection menu.	

The program should allow you to insert a number in between index 0 to number of numbers in the list.	

```
The list is:
197 159 70 139 194 60 26 195 87 146 188 45 46 11 30 93 43 24 194 142


Please select one from of the following selection or enter -1 to exit:
1. Insert an element to the List.
2. Delete an element from the List.
3. Repopulate the List information
:1

Enter the index where you want the new value to appear:
 0
Enter the value you want added at this location: 333
The list is:
333 197 159 70 139 194 60 26 195 87 146 188 45 46 11 30 93 43 24 194 142


Please select one from of the following selection or enter -1 to exit:
1. Insert an element to the List.
2. Delete an element from the List.
3. Repopulate the List information
:1

Enter the index where you want the new value to appear:
 21
Enter the value you want added at this location: 333
The list is:
333 197 159 70 139 194 60 26 195 87 146 188 45 46 11 30 93 43 24 194 142 333


Please select one from of the following selection or enter -1 to exit:
1. Insert an element to the List.
2. Delete an element from the List.
3. Repopulate the List information
:1

Enter the index where you want the new value to appear:
 5
Enter the value you want added at this location: 333
The list is:
333 197 159 70 139 333 194 60 26 195 87 146 188 45 46 11 30 93 43 24 194 142 333



Please select one from of the following selection or enter -1 to exit:
1. Insert an element to the List.
2. Delete an element from the List.
3. Repopulate the List information
:_
```

You can randomly delete any number on the list.	 The list is: 46 107 199 11 194 93 148 126 105 112 54 74 4 144 137 10 75 128 1 164 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 46 46 is deleted from the list. The list is: 107 199 11 194 93 148 126 105 112 54 74 4 144 137 10 75 128 1 164 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 164 164 is deleted from the list. The list is: 107 199 11 194 93 148 126 105 112 54 74 4 144 137 10 75 128 1 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 11 11 is deleted from the list. The list is: 107 199 194 93 148 126 105 112 54 74 4 144 137 10 75 128 1 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 137 137 is deleted from the list. The list is: 107 199 194 93 148 126 105 112 54 74 4 144 10 75 128 1 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :
After we deleted all the number on the list, the screen should display "The list is empty" information.	 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 31 31 is deleted from the list. The list is: 146 135 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 146 146 is deleted from the list. The list is: 135 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 135 135 is deleted from the list. The list is empty Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :

When we delete a number but there is more than one occurrence of the number in the list, the lower order (index) number will be deleted.	 The list is: 85 155 145 174 64 104 157 170 101 108 49 133 60 143 10 21 36 140 145 108 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 145 145 is deleted from the list. The list is: 85 155 174 64 104 157 170 101 108 49 133 60 143 10 21 36 140 145 108 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :
When we delete an item but there is no occurrence in the list, there should show "The number is not found." Information.	 Enter the index where you want the new value to appear: 5 Enter the value you want added at this location: 333 The list is: 333 197 159 70 139 333 194 60 26 195 87 146 188 45 46 11 30 93 43 24 194 142 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 111 The number is not found. Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :

At any point, a user can choose third option of the menu item to generate a new 20 numbers list.	 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :1 Enter the index where you want the new value to appear: 5 Enter the value you want added at this location: 333 The list is: 333 197 159 70 139 333 194 60 26 195 87 146 188 45 46 11 30 93 43 24 194 142 33 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :2 Enter an integer to delete: 111 The number is not found. Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :3 The list is: 13 112 156 83 15 122 18 157 187 113 138 16 121 144 15 16 19 102 69 122 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :
At the menu selection, if a user chooses a number other than -1, 1, 2 and 3, the program should display an error message and allows the user to reenter a selection.	 The list is: 13 112 156 83 15 122 18 157 187 113 138 16 121 144 15 16 19 102 69 122 Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :5 The input information is not a selection. Please input the selection again. Please select one from of the following selection or enter -1 to exit: 1. Insert an element to the List. 2. Delete an element from the List. 3. Repopulate the List information :

Please document CISP400V10AD7.cpp, ListNode.h and List.h files properly and zip them into a proper named zip file for an advance assignment (refer to the assignment section of the class syllabus) and submit it to the A7 folder of the Canvas Website.

Worth 180 points